

GLA UNIVERSITY, MATHURA

Department of Computer Science & Engineering

Introduction to Web Development

ASSIGNMENT 1

Name: Aditya Singh

Roll No: 5

Section: CS

College: GLA University, Mathura

Q1. Difference between Frontend, Backend, and Full-Stack Development

Frontend development deals with everything a user sees and interacts with directly in the browser — layout, design, buttons, forms, and animations. It is built using HTML, CSS, and JavaScript, along with frameworks like React or Angular. For example, the visual product listing page and shopping cart interface on Amazon is frontend work.

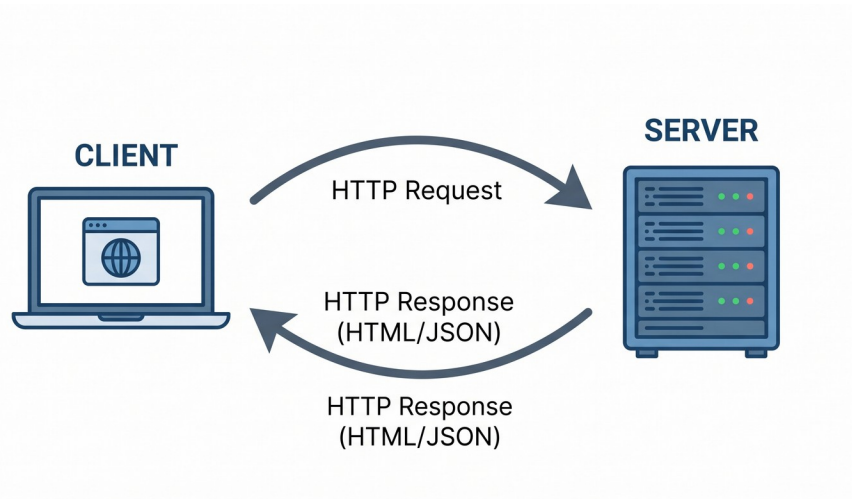
Backend development handles the server side of an application — business logic, database operations, authentication, and APIs. It runs on the server and is invisible to the user. Languages like Node.js, Python (Django/Flask), Java, or PHP are commonly used. For example, when Amazon processes a payment or checks stock in its database, that is backend work.

Full-stack development refers to working on both the frontend and backend of an application. A full-stack developer can design the user interface as well as build the server logic and database that power it. For example, a developer who builds both the UI of a food-delivery app and the backend that tracks orders and restaurants is a full-stack developer.

Q2. Diagram: Client-Server Model in Web Architecture

The client-server model works as follows: the client (browser or app) sends an HTTP request to the server for a resource (a web page, image, or data). The server receives the request, processes it (sometimes fetching data from a database), and sends back an HTTP response, usually as HTML, JSON, or a file. The client then renders this response for the user to view.

Basic flow: Client → Request (HTTP) → Server → Processing → Response (HTML/JSON) → Client (renders page).



Q3. How a Browser Requests and Displays a Web Page

When a user types a URL in the browser, the browser first performs a DNS lookup to convert the domain name into an IP address. It then establishes a TCP connection with the web server and sends an HTTP/HTTPS request for the requested page.

The web server processes this request — it may fetch data from a database or simply locate the requested file — and sends back a response containing HTML, CSS, JavaScript, and other assets (images, fonts). The browser's rendering engine parses the HTML to build the DOM, applies CSS to build the render tree (CSSOM), and executes JavaScript to add interactivity. Finally, the browser paints the page pixel by pixel on the screen for the user to see.

Q4. Tools Required to Set Up a Web Development Environment

- Code Editor / IDE (e.g., VS Code): Used to write and edit HTML, CSS, and JavaScript code with features like syntax highlighting and extensions.
- Web Browser (e.g., Chrome, Firefox): Used to run and preview the website, and to debug it using built-in Developer Tools.
- Version Control System (e.g., Git): Used to track changes in code and collaborate with a team without overwriting each other's work.
- Node.js and npm: Provide a JavaScript runtime outside the browser and a package manager to install libraries/frameworks.
- Web Server (Local, e.g., XAMPP/Live Server extension): Used to test dynamic pages and simulate a real hosting environment locally.
- Database Software (e.g., MySQL, MongoDB): Used to store and manage application data.
- Terminal/Command Line: Used to run commands, install packages, and manage the project through Git and npm.

Q5. What is a Web Server? Examples

A web server is software (often running on dedicated hardware) that stores, processes, and delivers web pages to clients over the internet using the HTTP/HTTPS protocol. It listens for incoming requests from browsers, retrieves or generates the requested content, and sends it back as a response.

Commonly used web servers include Apache HTTP Server, Nginx, Microsoft IIS (Internet Information Services), and LiteSpeed. Apache and Nginx are widely used for hosting websites and applications due to their reliability, performance, and support for various programming languages.

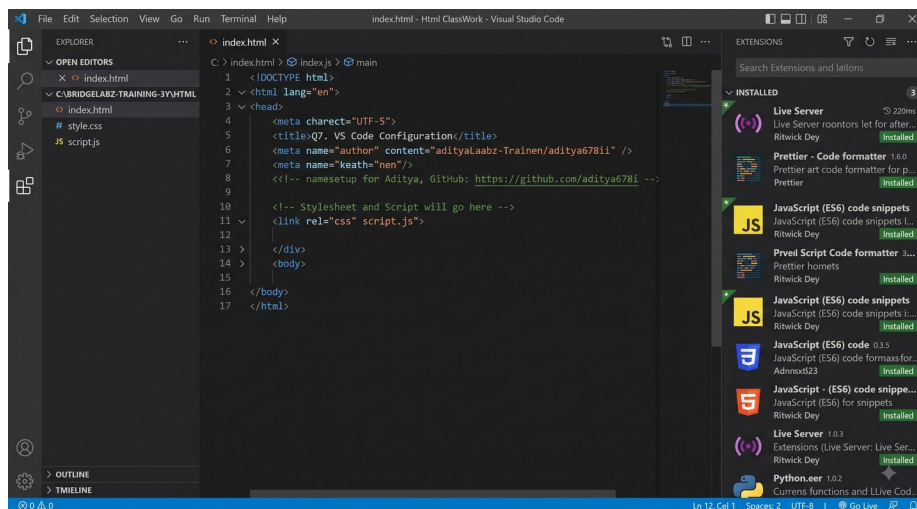
Q6. Roles: Frontend Developer, Backend Developer, Database Administrator

- Frontend Developer: Designs and builds the user interface and user experience of the website using HTML, CSS, and JavaScript; ensures the site is responsive, accessible, and visually appealing.

- Backend Developer: Builds the server-side logic, APIs, and application workflows; connects the frontend to the database and handles authentication, security, and data processing.
- Database Administrator (DBA): Designs, manages, and maintains the database; ensures data integrity, performs backups, optimizes queries, and manages access permissions and security.

Q7. Install and Configure VS Code for HTML, CSS, and JavaScript

Steps: (1) Download VS Code from code.visualstudio.com for your operating system and install it. (2) Open VS Code and go to the Extensions panel (Ctrl+Shift+X). (3) Install helpful extensions such as 'Live Server' (to preview HTML pages with auto-reload), 'Prettier' (for code formatting), and 'HTML CSS Support' / 'JavaScript (ES6) code snippets'. (4) Create a project folder with an index.html, style.css, and script.js file. (5) Right-click index.html and select 'Open with Live Server' to preview the page in the browser.



Q8. Static vs Dynamic Websites

A static website displays the same fixed content to every visitor. It is built using only HTML and CSS, does not interact with a database, and the content changes only when a developer manually edits the code. For example, a simple portfolio or brochure website with fixed pages is static.

A dynamic website generates content on the fly based on user interaction, data from a database, or server-side logic, so different users may see different content. It uses server-side languages (like PHP, Node.js, Python) along with a database. For example, an e-commerce site like Amazon or a social media site like Instagram is dynamic, since content such as product listings or the news feed changes for each user and each visit.

Q9. Five Web Browsers and Their Rendering Engines

- Google Chrome — uses the Blink rendering engine (a fork of WebKit), known for speed and broad extension support.

- Mozilla Firefox — uses the Gecko rendering engine, known for strong support of open web standards and privacy features.
- Apple Safari — uses the WebKit rendering engine, optimized for macOS and iOS devices with good battery efficiency.
- Microsoft Edge — uses the Blink engine (since it moved to a Chromium base in 2020), similar rendering behaviour to Chrome.
- Opera — also uses the Blink engine (Chromium-based), with additional built-in features like a free VPN and ad blocker.

Because Chrome, Edge, and Opera all use the Blink engine, pages usually render very similarly across them, whereas Firefox (Gecko) and Safari (WebKit) may render CSS features, fonts, or JavaScript APIs slightly differently, which is why developers test websites across multiple browsers for cross-browser compatibility.

Q10. Labeled Diagram: Basic Web Architecture Flow

The basic web architecture flow involves four main components: the Client sends a request to the Server; the Server processes the request and may call an API for specific functionality or communicate with a Database to fetch/store data; the Database returns the requested data to the Server; and finally, the Server sends the response back to the Client, which renders and displays it to the user.

Flow: Client → (HTTP Request) → Server → (Query) → Database → (Data) → Server → (via API, if needed) → Server → (HTTP Response) → Client.

